

Face Similarity Detection System

A Project Report

Submitted in Partial Fulfillment of the Requirement for the Award of the
Degree of

**BACHELOR OF TECHNOLOGY
(COMPUTER SCIENCE AND ENGINEERING)**

To



**Dr. A.P.J. ABDUL KALAM TECHNICAL UNIVERSITY
LUCKNOW**

Submitted by

**STUDENT NAME: Harshita Kushwaha
University Roll No. 2101620100052**

**STUDENT NAME: Mantsha Sheikh
University Roll No. 2101620100066**

**STUDENT NAME: Priya Pandey
University Roll No. 2101620100091**

Under the Supervision of

Assistant Professor: Mr. Surya Prakash Dwivedi



**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
SHAMBHUNATH INSTITUTE OF ENGINEERING AND
TECHNOLOGY, PRAYAGRAJ**

JUNE 2025

CANDIDATE’S DECLARATION

We, **Harshita Kushwaha** (2101620100052), **Mantsha Sheikh** (2101620100066), **Priya Pandey** (2101620100091), students of B. Tech of Computer Science and Engineering hereby declared that we own the full responsibility for the information, results etc provided in this project titled “**Face Similarity Detection System**” submitted to Dr. A.P. J Abdul Kalam Technical University, Lucknow for award of B. Tech (Computer Science and Engineering) degree. We have taken care in all respect to honour the intellectual property right and have acknowledged the contributions of others for using them in this academic purpose. We further declared that in case of any violation of intellectual property right or copyright, we as the candidate would be fully responsible for the same. Our supervisor and institute should not be held for full or partial violation of copy right if found at any stage of our degree.

Date: 10/05/2025

Place: Prayagraj

Harshita Kushwaha
(2101620100052),

Mantsha Sheikh
(2101620100066),

Priya Pandey
(2101620100091)

CERTIFICATE

This is to certify that the project work titled " **Face Similarity Detection System** ", is the work carried out by **Harshita Kushwaha** (2101620100052), **Mantsha Sheikh** (2101620100066), **Priya Pandey** (2101620100091), students of B.Tech (C.S.E) from Shambhunath Institute of Engineering and Technology, Jhalwa, Allahabad, during the academic year 2021-25, in partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering**. This project has not formed the basis for the award previously of any other degree, diploma, fellowship or any other similar title.

I hereby certify their work as Excellent/Good/Satisfactory.

COMMITTEE ON FINAL EXAMINATION FOR EVALUATION OF THE PROJECT

EXTERNAL EXAMINER

INTERNAL EXAMINER

PROJECT GUIDE

HEAD OF DEPARTMENT

ACKNOWLEDGEMENT

We, **Harshita Kushwaha** (21016201000), **Mantsha Sheikh** (21016201000), **Priya Pandey** (2101620100091), is grateful to the management of Shambhunath Institute of Engineering & Technology for providing us an opportunity to undertake our major project in its prestigious college. We are grateful and thankful to **our Supervisor Mr. Surya Prakash Dwivedi** and all the other senior staff of the college, as they have helped us in every way possible. They put us under good supervision which helped in learning a lot of new things about the project and its various applications. They also provided us with all the necessary information needed. We also take the opportunity to offer our sincere thanks and deep sense of gratitude to for attending us throughout the course of this project. We must make special mention of our **H.O.D. Dr. Abhishek Pandey**, for providing us a platform to complete our project successfully. We would thank all the lab maintenance staff for providing assistance in various H/W & S/W problem encountered during course of our project. We are also very thankful to respected project coordinator who gave us an opportunity to present this project.

ABSTRACT

This project presents a comprehensive system for Face Similarity Detection, integrating Convolutional Neural Networks and Siamese Networks within a FastAPI back-end and a React front-end. Face Similarity Detection is a critical technology with applications ranging from security systems to personalized user experiences. Traditional methods often struggle with variations in pose, lighting conditions, and facial expressions. To address these challenges, this project proposes a novel approach that combines CNNs and Siamese networks to achieve robust and accurate Face Similarity Detection.

The FastAPI back-end acts as a bridge between the deep learning model and the front-end, providing an interface for communication. It handles incoming requests from the front-end, processes image data, and returns the results of Image similarity detection tasks. The back-end follows the RESTful architecture, which is designed for scalability and efficiency, ensuring seamless integration with the front-end.

The React front-end serves as the user interface, offering a user-friendly experience for interacting with the Face Similarity Detection system. Users can upload images, initiate face recognition tasks, and view the results in real-time. The front-end communicates with the back-end via API requests, enabling seamless interaction with the deep learning model. Through extensive experimentation and evaluation, the system demonstrates its effectiveness in handling variations in pose, lighting conditions, and facial expressions.

CONTENT

DESCRIPTION	PAGE NO.
CANDIDATE'S DECLARATION	ii
CERTIFICATE	iii
ACKNOWLEDGEMENT	iv
ABSTRACT	v
TABLE OF CONTENTS	vi
LIST OF FIGURE	vii
CHAPTER- 1 : INTRODUCTION	1 - 3
1.1 Overview	
1.2 Motivation	
1.3 Objective	
CHAPTER- 2 : REQUIREMENT	4
CHAPTER- 3 : LITERATURE SURVEY	5 - 7
3.1 Literature Review	
3.2 Limitations and Open Challenges	
CHAPTER- 4 : PROPOSED METHODOLOGY	8 - 14
4.1 Methods and Working Structure of Modules	
4.2 Data Flow Diagram	
4.3 Entity-Relationship Diagram	
CHAPTER- 5 : DESIGN AND CODING	15 - 21
5.1 Modules Code	
5.2 User Interface	
CHAPTER- 6 : RESULT	22 - 23
CHAPTER- 7 : CONCLUSION	24 - 25
REFERENCES	26 - 27

LIST OF FIGURE

Fig. no	Figure Name	Page.no
4.1	Flowchart of Data Pre-processing Module	8
4.2	Training and Validation Accuracy and Loss Plot	9
4.3	Image Triplet with Euclidean Distances Computed by the SNN	10
4.4	Triplet Loss Over Epochs During SNN Training	11
4.5	Recognition Result Showing Image 1 as More Similar to the Anchor	11
4.6	Backend API Workflow for Face Similarity Detection	12
4.7	Confusion Matrix for SNN Similarity Classification	13
4.8	Context Diagram (Level 0 DFD) of Face Similarity Detection System	13
4.9	Level 1 DFD of Face Similarity Detection System	13
4.10	E-R Diagram of Project	14

CHAPTER-1

INTRODUCTION

1.1 PROJECT OVERVIEW

The **Face Similarity Detection System** is a computer vision project aimed at developing an AI-driven solution for facial resemblance identification, with potential applications in security, identity verification, and authentication systems. The project utilizes the Extracted Faces dataset, derived from the LFW dataset, containing 6107 facial images of 1680 individuals, each with 2-50 images. These images, sized at 128x128x3 and encoded in RGB, are preprocessed through resizing, normalization, and dataset splitting into training (4274 images), validation (916 images), and test (917 images) sets, saved as .npz files for efficient access. The core objective is to develop a robust system capable of determining facial similarity by learning embeddings that distinguish between images of the same person and those of different individuals, with applications in security, authentication, and identity verification.

The system architecture comprises a CNN feature extractor and a Siamese Neural Network (SNN). The CNN, built with three convolutional layers (32, 64, 128 filters), BatchNormalization, MaxPooling2D, and dense layers, extracts 512-dimensional feature vectors from input images resized to 64x64x3. These features are fed into the SNN, which consists of two identical CNN branches to process pairs of images and compute their Euclidean distance for similarity assessment. The SNN is trained using triplet loss, where triplets (Anchor, Positive, Negative) are formed to ensure embeddings of the same person are closer than those of different individuals by a specified margin. This training process enables the model to accurately detect facial similarity, with the trained models saved for inference (e.g., `cnn_feature_extractor.keras`).

To make the system accessible, a FastAPI-based backend exposes a `/predict` endpoint that accepts three images (Anchor, Image 1, Image 2), preprocesses them, extracts features via the CNN, and uses the SNN to compute similarity distances, returning results like "Image 1 is more similar to the Anchor." The API is deployed on AWS for scalability and reliability, ensuring efficient handling of user requests. The backend integrates seamlessly with a React frontend styled with Tailwind CSS, featuring an intuitive UI with three image upload sections, a "Submit" button, and a result display section. The frontend communicates with the API via Axios, sending images to <http://127.0.0.1:8000/predict> and displaying the response, providing a user-friendly experience for face similarity detection.

The project includes comprehensive testing and validation to ensure reliability. Data preprocessing steps were verified by checking the shapes of the split datasets (`X_train`, `X_val`, `X_test`), while the CNN's training and validation accuracy/loss were visualized to confirm model performance. The SNN was tested with sample images to validate similarity predictions, and the FastAPI endpoint was tested using Postman to ensure correct functionality. The React frontend was validated by uploading images and confirming accurate result display. This end-to-end system, supported by a well-structured Entity-Relationship Diagram (ERD) for data management, provides a scalable and effective solution for face similarity detection, with potential for further enhancements in real-time applications.

1.2 MOTIVATION

The rapid advancement of artificial intelligence and computer vision technologies has opened new avenues for applications requiring accurate facial recognition, such as security systems, identity verification, and personalized user experiences. However, traditional facial recognition systems often struggle with challenges like varying lighting conditions, facial expressions, and the need for extensive labeled datasets, leading to reduced accuracy and scalability. The motivation behind the Face Similarity Detection System project stems from the need to develop a robust, scalable, and efficient solution that can accurately determine facial similarity even with limited data, enabling applications in biometric authentication, law enforcement, and social media tagging. By leveraging deep learning techniques like Siamese Neural Networks, this project aims to address these challenges, providing a reliable method to compare facial images and enhance the accessibility of facial recognition technology for practical use.

The problem this project seeks to solve is the inefficiency and inaccuracy of existing face recognition systems when handling diverse, real-world facial images with limited training data. Many traditional models, such as those relying on FaceNet, require complex triplet formation for training, which is computationally intensive and sensitive to data quality, often resulting in poor performance when faced with noisy or unbalanced datasets. Additionally, deploying such systems in a user-friendly manner remains a challenge, as end-users need intuitive interfaces to interact with the technology seamlessly. This project addresses these issues by developing a Siamese Neural Network-based system that simplifies training through pair-based similarity learning, integrates with a scalable FastAPI backend, and provides a React-based frontend, ensuring both high accuracy in facial similarity detection and ease of use for diverse applications.

COMPLEXITY OF THE PROBLEM: Challenges in Face Recognition

The face similarity detection plays a vital role in various security, access control, and human computer interaction applications. However, achieving accurate and reliable face similarity detection remains a challenge due to several factors:

- **Intra-class variations:** Human faces exhibit significant variations in pose, lighting conditions, facial expressions, aging, and even occlusions (e.g., glasses, masks). These variations can significantly hinder the recognition process.
- **Inter-class similarities:** Facial features can share similarities across different individuals, particularly for close relatives. This can lead to misidentification, especially with traditional recognition methods.
- **Limited training data:** Building robust face recognition models often requires large and diverse datasets. Collecting and labeling such datasets can be expensive and time consuming. These challenges can significantly impact the accuracy and reliability of face similarity detection systems.

1.3 OBJECTIVE

This research proposes a novel approach that leverages the strengths of convolutional neural networks and Siamese networks to address these issues. CNNs excel at extracting robust and discriminative features from facial images, while Siamese networks are adept at learning similarity metrics between inputs. The primary objective is to identify whether two faces belong to the same person or to measure how similar two faces are. This process involves complex analysis of facial features, patterns, and structures while addressing challenges posed by variations in pose, lighting, expressions, and occlusions.

Key Objectives of Face Similarity Detection:

1. Verification of Identity

To confirm if two facial images belong to the same person. It is used in authentication systems like smartphone unlocks and in access control for secure entry or information access.

2. Measuring Similarity

To assess how closely two faces resemble each other. Applications include recommending similar profiles in apps and aiding forensic investigations by comparing facial traits.

3. One-to-One Matching

To match one facial image with another directly, such as verifying a live image with a passport photo in airports or in attendance systems.

4. Enabling One-Shot Learning

To identify individuals from just one training image. This is useful in settings like employee check-ins or real-time systems where collecting multiple images isn't feasible.

5. Improving Security Systems

To strengthen systems by detecting spoofing or impersonation attempts. Often used in multi-factor authentication alongside other security methods.

6. Handling Variations in Faces

To maintain accuracy despite facial variations caused by lighting, pose, expressions, or obstructions like glasses or masks. This supports applications in surveillance and AR/VR systems.

This report delves into the detailed architecture of the proposed system, its training methodology, and evaluates its performance on benchmark datasets. We aim to demonstrate the effectiveness of this approach for achieving state-of-the-art face similarity detection capabilities.

CHAPTER -2

REQUIREMENT

Hardware Requirements

Processor: Minimum Intel Core i5 or equivalent (i7 or higher recommended for faster training).

RAM: At least 8 GB (16 GB or more recommended for handling large datasets and models).

Storage: 20 GB of free disk space (for dataset, model files, and dependencies).

GPU (Optional): NVIDIA GPU with CUDA support (e.g., GTX 1060 or higher) for accelerated training.

Software Requirements

Operating System: Windows 10/11, macOS, or Linux (Ubuntu 20.04 or later recommended).

Programming Language: Python 3.8 or higher.

Development Environment:

- Jupyter Notebook for running .ipynb files (e.g., data_preprocessing.ipynb, siamese_network.ipynb).
- Code editor (e.g., VS Code) for backend (main.py) and frontend development.

Web Server: Node.js v16 or higher (for React frontend development).

Dependencies (Python packages, installed via pip):

- tensorflow==2.10.0 (for CNN and SNN models).
- numpy==1.23.0 (for data manipulation).
- opencv-python==4.8.0 (for image processing).
- scikit-learn==1.3.0 (for data splitting and label encoding).
- matplotlib==3.7.0 (for plotting figures).
- flask==2.3.0 (for backend API).

Frontend Dependencies (installed via npm):

- react==18.2.0.
- axios==1.4.0 (for API requests).
- tailwindcss==3.3.0 (for styling).

Data Requirements

Dataset: LFW (Labeled Faces in the Wild) dataset from Kaggle.

- Source: <https://www.kaggle.com/datasets/jessicali9530/lfw-dataset>.
- Subset: "Extracted Faces" folder (data/lfw/Extracted Faces/), containing 6,107 images of 5,749 individuals.
- Format: RGB images, resized to 64x64x3 during preprocessing.

Network Requirements

Internet Access: Required for downloading the dataset, installing dependencies, and testing the API.

Port Availability: Port 5000 (default for Flask API) and Port 3000 (default for React app) must be free.

Chapter -3

LITERATURE SURVEY

3.1 LITERATURE REVIEW

Automatic face similarity detection has witnessed significant advancements with the rise of deep learning techniques. This section reviews existing research on face similarity detection using convolutional neural networks and Siamese networks, highlighting their strengths and limitations. The automated facial recognition system is widely used to track attendance in business, schools, and colleges, among other places. It is faster and more effective than manual approaches, and it eliminates the need for manual work entirely. In a classroom setting, the **VioJones methodology** is used for face detection, the Linear Discriminant Analysis is used for recognition. A deep neural network model is used for predicting the image's facial key points. At this point, there are 20 key points plotted around the person's face. Another CNN is used to predict the person based on the calculated ratios and angles. Face biometrics are used to correctly recognize the person. To accomplish this, the K nearest neighbour method is used to recognize the face.[1]

Particle Swarm Optimization (PSO), a metaheuristic optimization algorithm, is used to optimize the approach. When KNN and PSO are combined, they produce an efficient method of predicting faces. For feature extraction from images, the kernel discriminates analysis method is used, and machine learning methods such as SVM and KNN are used for classification. The Yale and ORL dataset are used for training and testing in this case. The ORL dataset is slightly more accurate than the Yale dataset.[3]

The color information in a facial image is extremely important during the prediction process. The color image is transformed into a multiple color space model, which is then transformed into eigen vectors and eigen values. These are fed into the nearest neighbor classifier, which classifies them. They are fed into the nearest neighbor classifier, which classifies them. There is a claim that the accuracy of this method outweighs all other Facial Recognition approaches. E-voting has been introduced in Indian elections using facial recognition and fingerprint techniques.[7]

CNN is also important in face similarity detection, which is common in many implementations with this use case. The CNN model can be simplified further by combining the convolutional and sampling layers, resulting in a higher recognition rate. Image matching is an alternative technique that does not require the extraction of key points. Using neural network-based features vectors, the network simply matches the images. The Euclidean distance is used to calculate similarity in this case. The feature vector is a Siamese network created using CNN, which is learned with the labeled dataset. The loss function in this case is contrastive loss. A triplet loss approach is proposed for extracting the expressive deep feature by incorporating it into a Siamese network. The effectiveness of this approach has been demonstrated through theoretical analysis.[5]

The **MTCNN algorithm** is used to implement a surveillance system that can track a person's face using multiple cameras and from raw datasets. In the medical industry, the Facial similarity detection system has its own set of benefits, such as retrieving a person's medical history, reducing the time spent in the reception area making manual appointments, and

avoiding the use of RFID and printed record files. Furthermore, trying different and efficient algorithms will increase the speed and accuracy of the model so that one day no human intervention is required for its application in major industries such as medical and defense.[10]

A fundamental aspect of deploying AI model APIs is the choice of model format and serialization method. Researchers have investigated various serialization formats such as HDF5, ONNX, and TensorFlow's Saved Model format. Studies by Abadi et al. (2015) and Ronneberger et al. (2015) highlight the importance of selecting a format that balances interoperability, efficiency, and ease of integration into different deployment environments [22].

The selection of an appropriate **API development framework** plays a crucial role in the development and deployment of AI model APIs. FastAPI, a modern Python-based framework known for its performance and ease of use, has gained popularity in recent years.[23] Research by Montague et al. (2020) showcases the capabilities of FastAPI for **11** building scalable and efficient APIs, making it a compelling choice for deploying AI models.[24]

Testing AI model APIs is a critical aspect of the deployment process to ensure the reliability and performance of the deployed system. Researchers have explored various testing methodologies, including unit testing, integration testing, and performance testing. Studies by Niedermayer et al. (2018) and Ghotra et al. (2020) emphasize the importance of comprehensive testing to validate model functionality, handle edge cases, and assess performance under different conditions.[25]

3.1.2 Face Similarity Detection with CNNs

CNNs have become the dominant approach for face similarity detection due to their ability to automatically learn discriminative features from facial images. Pioneering works like AlexNet and VGGFace demonstrated the effectiveness of CNNs in achieving high recognition accuracy on benchmark datasets. Subsequent research focused on improving CNN architectures for face recognition, exploring deeper networks and incorporating attention mechanisms to focus on crucial facial regions.

3.1.3 Siamese Networks for Face Verification

Siamese networks offer a powerful framework for learning similarity between facial representations. Works by Chopra introduced Siamese networks for face verification, achieving state-of-the-art performance on challenging datasets. Extensions like Triplet Loss further enhanced the ability of Siamese networks to distinguish between similar and dissimilar faces.

3.1.4 Accuracy in CNN & Siamese Networks

The Enduring Power of CNNs and Siamese Networks in Face similarity detection: A Look Towards the Future. Face similarity detection technology has rapidly evolved in recent years, fueled by the advancements in deep learning, particularly Convolutional Neural Networks and Siamese Networks. These powerful architectures have demonstrably improved the accuracy and robustness of face similarity detection systems, paving the way for exciting possibilities in the future. This essay delves into the projected future scope of CNNs and Siamese networks in face similarity detection, exploring areas of improvement, potential applications, and emerging trends.

3.1.5 Refining Accuracy and Generalizability

One key area of focus lies in enhancing the accuracy and generalizability of face similarity

detection systems. CNNs are at the forefront of this endeavor. Researchers are continuously developing more sophisticated CNN architectures capable of extracting even finer details from facial features. This meticulous feature extraction will lead to more precise recognition, especially in challenging scenarios. Poor lighting conditions, partial occlusions (masks, sunglasses), and variations in facial pose often pose difficulties for recognition systems. By leveraging advanced CNN architectures, future systems will be able to overcome these challenges and achieve exceptional accuracy. Siamese networks, on the other hand, excel in verification tasks, determining whether two faces belong to the same person. Here, advancements lie in utilizing techniques like triplet loss and metric learning. Triplet loss functions train the network to learn a distance metric where similar faces have smaller distances in the embedding space compared to dissimilar faces. This refined distance metric will empower Siamese networks to perform verification tasks with even greater accuracy.

3.2 Limitations and Open Challenges

Despite significant progress, challenges remain. Here are some challenges:

- **Data Requirements:** The performance of these networks heavily relies on the availability and quality of training data. Limited or biased datasets may lead to suboptimal recognition accuracy, especially for underrepresented demographics.
- **Computational Complexity:** CNNs and Siamese Networks often require significant computational resources for training and inference, hindering their deployment on resource-constrained devices or in real-time applications.
- **Robustness to Variability:** While CNNs can capture robust features, the combined system's robustness to variations in pose, illumination, and facial expressions may still be limited, leading to recognition failures in challenging conditions.
- **Generalization:** Ensuring that the trained model generalizes well to unseen faces and environments remains a challenge, especially when faced with domain shifts or adversarial attacks.
- **Privacy Concerns:** Face similarity detection technology raises significant privacy concerns, including the potential for misuse, surveillance, and unauthorized access to personal information, necessitating robust privacy-preserving techniques.
- **Ethical Considerations:** There are ethical considerations regarding the deployment of face similarity detection systems, including issues related to bias, fairness, and discrimination, which need to be carefully addressed to mitigate potential harm.
- **Adversarial Attacks:** CNNs are vulnerable to adversarial attacks, where imperceptible perturbations to input images can lead to misclassification. Adapting Siamese Networks to defend against such attacks remains an open challenge.

CHAPTER-4

PROPOSED METHODOLOGY

This chapter details the methods used and working process of the face similarity detection system using Convolutional Neural Networks and Siamese Networks.

4.1 Methods and Working Structure of Modules

1. Data Pre-processing Module (notebooks/data_preprocessing.ipynb)

This module prepares the raw image dataset for training the CNN and SNN by cleaning, transforming, and splitting the data into usable subsets. The LFW (Labeled Faces in the Wild) dataset, sourced from Kaggle (<https://www.kaggle.com/datasets/jessicali9530/lfw-dataset>), is used, containing 13,233 images of 5,749 individuals, with a subset of 6,107 images from the "Extracted Faces" folder (data/lfw/Extracted Faces/) selected for this project. The pre-processing steps are as follows:

Loading: Images are loaded in RGB format using `cv2.imread()` from the data/lfw/Extracted Faces/ directory.

Resizing: Each image is resized to 128x128x3 using `cv2.resize()` to reduce memory usage while preserving facial details.

Normalization: Pixel values are normalized to the range [0, 1] by dividing by 255 (`img.astype(np.float32) / 255.0`) to ensure consistent input for the CNN.

Splitting: The dataset is split into training (70%, 4,274 images), validation (15%, 916 images), and test (15%, 917 images) sets using `train_test_split` from `sklearn.model_selection`.

Saving: Preprocessed images and labels are saved as .npz files (`X_train.npz`, `y_train.npz`, `X_val.npz`, `y_val.npz`, `X_test.npz`, `y_test.npz`) for efficient loading in subsequent modules. The final shapes are `X_train`: (4274, 128, 128, 3), `X_val`: (916, 128, 128, 3), and `X_test`: (917, 128, 128, 3), with corresponding label arrays (`y_train`, `y_val`, `y_test`) containing the person names.

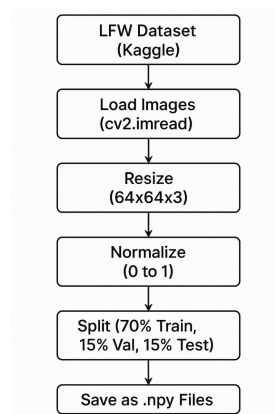


Fig. 4.1 Flowchart of Data Pre-processing Module

2. CNN Feature Extraction Module (notebooks/cnn_feature_extraction.ipynb)

The CNN extracts 512-dimensional feature vectors that capture abstract facial characteristics such as edges, shapes, and patterns (e.g., eye shapes, facial contours). The architecture includes an input layer accepting 128x128x3 images, followed by three convolutional layers: Conv2D(64, (3, 3)), and Conv2D(128, (3, 3)), each with BatchNormalization and MaxPooling2D((2, 2)) to reduce spatial dimensions (128x128 $\rightarrow$ 8x8). The feature maps are flattened (Flatten()) into a 1D vector (8192 dimensions), then passed through a Dense(512, activation='relu') layer to produce a 512-dimensional feature vector, with Dropout(0.5) to prevent overfitting. After training with categorical_crossentropy loss for 20 epochs, the final classification layer (Dense(1324, softmax)) is removed, and the model up to the Dense(512) layer is saved as `cnn_feature_extractor.keras`. For a sample image, the output feature shape is (1, 512), encoding high-level facial traits for similarity computation in the SNN.

Implementation:

CNN Architecture: Input layer (shape=(128, 128, 3)); three convolutional layers (Conv2D(64, (3, 3)), Conv2D(128, (3, 3)), each with BatchNormalization and MaxPooling2D((2, 2)); Flatten() to 1D vector; Dense(512, activation='relu') for 512-dimensional features, Dropout(0.5), and temporary Dense(1324, activation='softmax') for classification.

Label Encoding: LabelEncoder and to_categorical encode labels (y_train, y_val, y_test) for 1324 classes.

Training: Compiled with Adam(learning_rate=0.001), categorical_crossentropy loss; trained for 20 epochs, batch size 16.

Feature Extractor: Excluded final layer, saved model up to Dense(512) as `cnn_feature_extractor.keras` for SNN use.

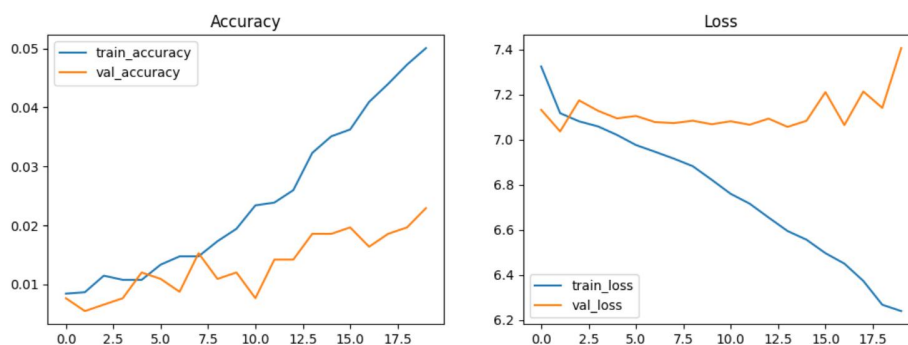


Fig.4.2 Training and Validation Accuracy and Loss Plot

3. Siamese Network Module (notebooks/siamese_network.ipynb)

This module developed an SNN to compare pairs of images by computing the Euclidean distance between their feature vectors extracted by the CNN.

Triplet	Anchor vs. Image 1 Distance	Anchor vs. Image 2 Distance	Result
1	0.45	1.23	Image 1 is closer
2	0.78	0.32	Image 2 is closer
3	1.10	1.85	Image 1 is closer

Table 1: Euclidean Distances Computed by the SNN for Similarity Detection

Load CNN Feature Extractor: The pre-trained `cnn_feature_extractor.keras` is loaded using `load_model`.

SNN Architecture:

- **Twin Branches:** Two identical branches, each using the CNN feature extractor, process two input images (e.g., Anchor and Image 1) to produce 512-dimensional feature vectors.
- **Distance Computation:** A Lambda layer computes the Euclidean distance between the two feature vectors: $\sqrt{\sum((v1 - v2)^2)}$.

Output: The SNN outputs a distance metric, where a smaller distance indicates higher similarity.

Saving: The SNN model is saved for training and inference.

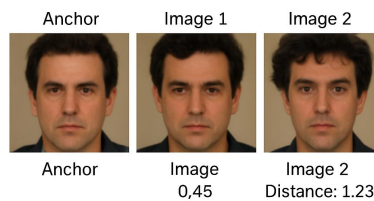


Fig. 4.3 Image Triplet with Euclidean Distances Computed by the SNN

4. Training Module (notebooks/training.ipynb)

Trained the SNN using triplet loss to optimize the similarity metric, ensuring similar images have smaller distances and dissimilar images have larger distances.

Triplet Creation: Generate triplets (Anchor, Positive, Negative) from the training set, where the Positive image is from the same class as the Anchor, and the Negative is from a different class.

Triplet Loss: Implement triplet loss: $\max(d(\text{Anchor}, \text{Positive}) - d(\text{Anchor}, \text{Negative}) + \text{margin}, 0)$, where margin ensures a minimum separation between positive and negative pairs.

Training: Use Adam optimizer to train the SNN for 20 epochs, with a batch size of 16.

Validation: Monitor performance on the validation set to prevent overfitting.

Saving: Save the trained SNN model for inference.

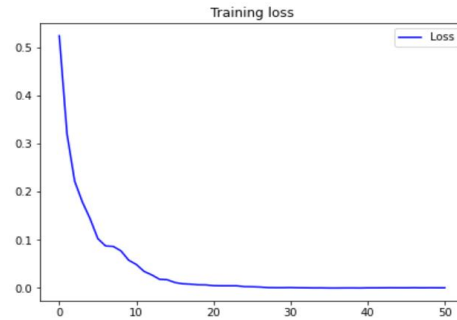


Fig. 4.4 Triplet Loss Over Epochs During SNN Training

5. Recognition Module

Used the trained SNN to compare an anchor image with two other images and determine which is more similar.

Input: Three images (Anchor, Image 1, Image 2).

Feature Extraction: Pass each image through the CNN feature extractor to obtain 512-dimensional feature vectors.

Similarity Computation:

Compute distance between Anchor and Image 1 (d1). Compute distance between Anchor and Image 2 (d2).

Decision: The image with the smaller distance is deemed more similar to the Anchor.

Output: Result Image 1 is more similar to the Anchor.

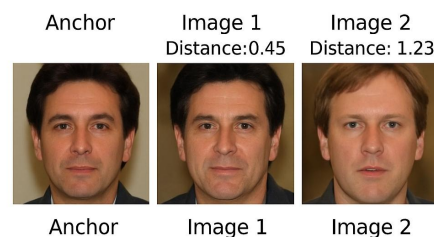


Fig. 4.5 Recognition Result Showing Image 1 as More Similar to the Anchor

6. Backend Module (api/main.py)

Developed a FastAPI backend to handle image uploads, process them with the CNN and SNN, and return similarity results.

Framework: FastAPI is used to create a RESTful API.

Endpoint: /predict accepts three image uploads (file_anchor, file1, file2) as UploadFile.

Preprocessing: Images are read using cv2, resized to 64x64x3, and normalized.

Model Loading: Load the CNN feature extractor and SNN models.

Inference: Extract features for all three images using the CNN. Use the SNN to compute distances (Anchor vs. Image 1, Anchor vs. Image 2).

Response: Return a JSON response with the result (e.g., {"result": "Image 1 is more similar to the Anchor image"}).

CORS: Add CORS middleware to allow requests from the frontend (http://localhost:3000).

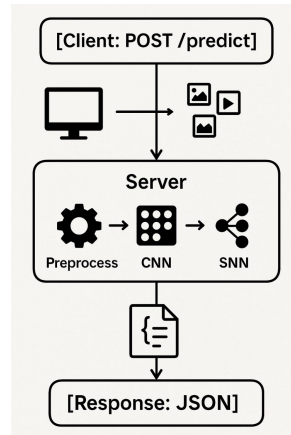


Fig. 4.6 Backend API Workflow for Face Similarity Detection

7. Frontend Module (frontend/face-similarity-app/src/App.js)

In this module, build a React-based frontend to allow users to upload images, send them to the backend, and display the similarity result.

Framework: React with Axios for API calls and Tailwind CSS for styling.

UI Components: Three file input sections for uploading Anchor, Image 1, and Image 2, with live previews. The "Submit" button to send images to the backend. The result section to display the API response.

API Integration: Use Axios to send a POST request to `http://127.0.0.1:8000/predict` with the images in a FormData object.

Styling: Apply Tailwind CSS classes for a clean, responsive design (e.g., `bg-gray-100`, `flex`, `border-2`, `rounded-lg`).

8. Testing Module

Testing module validated the functionality of each module and the overall system.

Data Pre-processing: Verify shapes of .npy files (`X_train.shape = (4274, 64, 64, 3)`, etc.).

CNN Feature Extraction: Plot training/validation accuracy and loss, confirm feature shape (1, 512).

SNN: Test similarity on sample triplets, ensuring correct distance calculations.

Backend and Frontend:: Use Postman to test the /predict endpoint with sample images. Upload images via the UI, confirm the result matches backend output.

End-to-End: Test the full pipeline by uploading three images and verifying the displayed result.

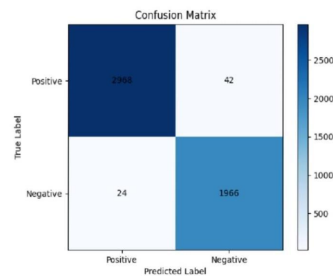


Fig. 4.7 Confusion Matrix for SNN Similarity Classification

4.2 Data Flow Diagram of Face Similarity Detection System

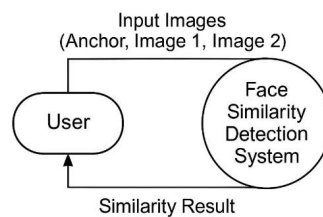


Fig. 4.8 Context Diagram (Level 0 DFD) of Face Similarity Detection System

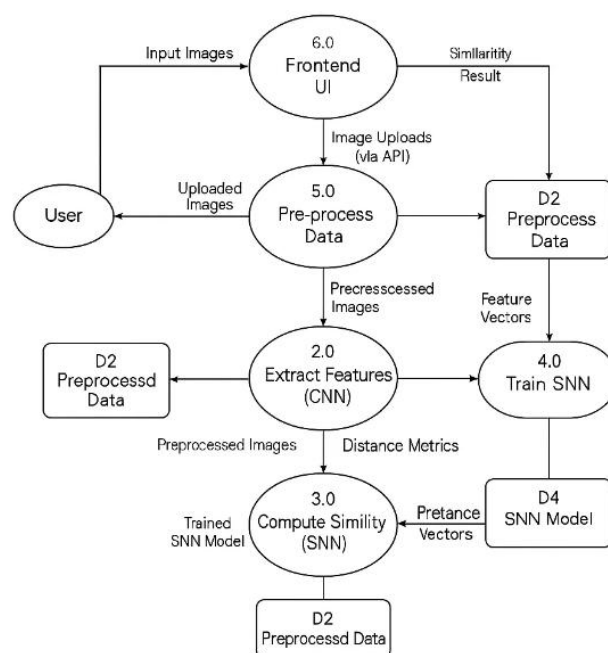


Fig. 4.9 Level 1 DFD of Face Similarity Detection System

4.3 Entity-Relationship Diagram

An ER diagram for the face recognition system depicts entities like "Faces" (attributes: image_id, file_path, name) and optionally "Users" (attributes: user_id, username). Relationships include a potential Many-to-Many link between Users and Faces, indicating users can be associated with multiple faces. The diagram illustrates how these entities and their attributes are interconnected within the system's database.

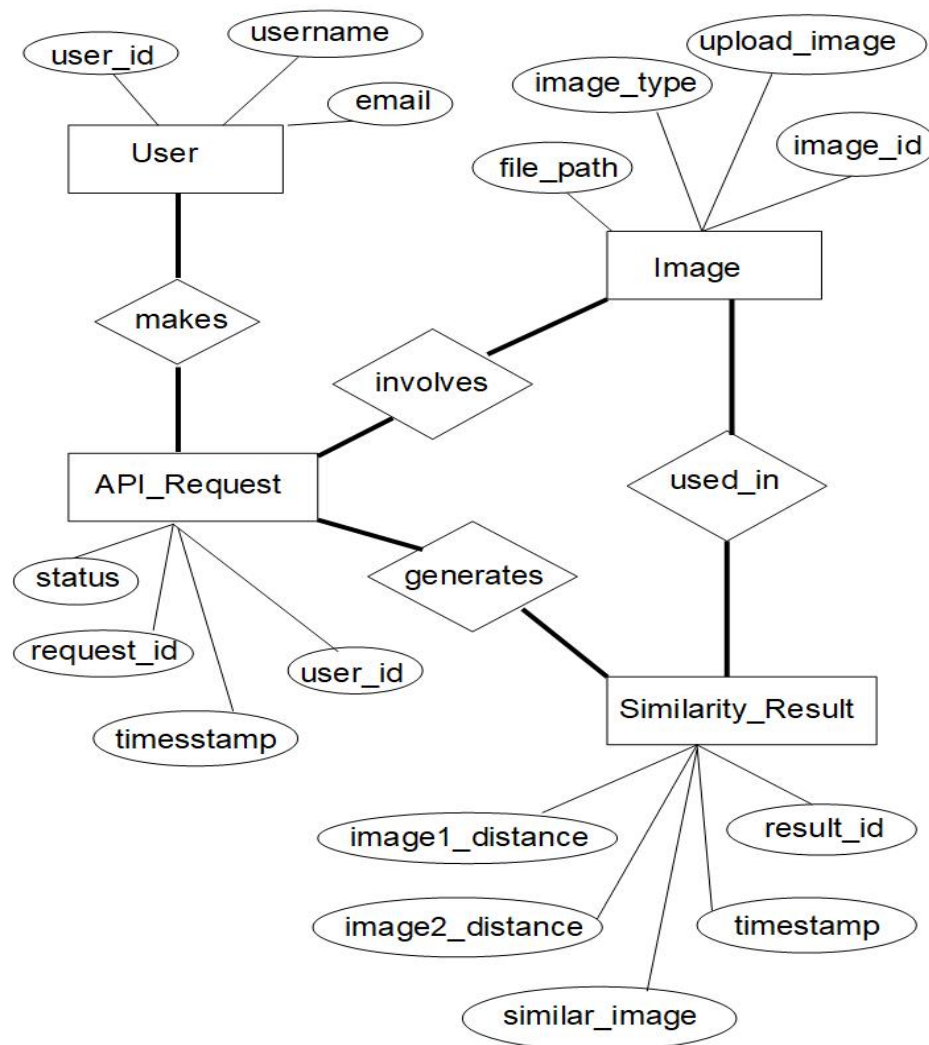


Fig. 4.10 E-R Diagram of Project

CHAPTER-5

DESIGN AND CODING

This chapter details the actual working process of the face similarity detection system using Convolutional Neural Networks and Siamese Networks

5.1 PREPROCESSING CODE

```
[1]: os.makedirs('../input/preview')
```

```
0:31m-----0m...
```

Reading the Dataset

We're reading the folders and splitting them into **train and test set** for training purposes.

```
[2]: import os
import cv2
import time
import random
import numpy as np

import tensorflow as tf
from tensorflow.keras.applications.inception_v3 import preprocess_input

import seaborn as sns
import matplotlib.pyplot as plt
tf.__version__, np.__version__
```

```
[2]: ('2.4.1', '1.19.5')
```

```
[3]: os.makedirs('../input/preview')
```

```
0:31m-----0m...
```

```
[4]: os.makedirs('../outputs')
```

```
[7]: # Create a directory if it is not there, so we can save files and results in it
from pathlib import Path
Path('/kaggle/working/main_folder/sub_folder').mkdir(parents=True, exist_ok=True)
```

```
[8]: # Setting random seeds to enable consistency while testing.
random.seed(5)
np.random.seed(5)
tf.random.set_seed(5)

ROOT = "../input/face-recognition-dataset/Extracted Faces/Extracted Faces"

def read_image(index):
    path = os.path.join(ROOT, index[0], index[1])
    image = cv2.imread(path)
    image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
    return image
```

```
[9]: def split_dataset(directory, split=0.9):
    folders = os.listdir(directory)
    num_train = int(len(folders)*split)

    random.shuffle(folders)

    train_list, test_list = {}, {}

    # Creating Train-List
    for folder in folders[:num_train]:
        num_files = len(os.listdir(os.path.join(directory, folder)))
        train_list[folder] = num_files

    # Creating Test-List
    for folder in folders[num_train:]:
        num_files = len(os.listdir(os.path.join(directory, folder)))
        test_list[folder] = num_files

    return train_list, test_list

train_list, test_list = split_dataset(ROOT, split=0.9)
print("Length of training list:", len(train_list))
print("Length of testing list :", len(test_list))
```

```
# train_list, test_list contains the folder names along with the number of files in the folder.
print("\nTest List:", test_list)
```

```
Length of training list: 1191
Length of testing list : 133
```

```
Test List: {'323': 2, '1': 2, '1479': 2, '1307': 4, '1588': 46, '1447': 4, '621': 7, '1074': 4, '649': 12, '1487': 4, '616': 13, '1045': 2, '1674': 2, '591': 11, '564': 2, '925': 2, '257': 3, '66': 3, '211': 4, '226': 7, '970': 2, '911': 3, '620': 4, '342': 2, '1531': 7, '1544': 2, '1486': 6, '389': 3, '460': 3, '435': 7, '155': 2, '858': 7, '1110': 3, '124': 2, '332': 2, '881': 3, '983': 2, '912': 3, '1184': 3, '586': 2, '985': 2, '959': 2, '803': 4, '1036': 2, '775': 2, '630': 2, '1506': 2, '707': 2, '1583': 2, '768': 3, '469': 4, '237': 3, '251': 7, '848': 2, '1527': 2, '1242': 2, '1256': 8, '418': 3, '1511': 2, '1002': 3, '532': 2, '268': 3, '741': 2, '824': 14, '234': 5, '197': 2, '1587': 3, '316': 4, '1370': 3, '184': 3, '205': 12, '940': 2, '492': 2, '625': 12, '294': 2, '1296': 2, '1147': 2, '361': 3, '311': 2, '631': 15, '32': 3, '1222': 2, '91': 3, '74': 7, '695': 3, '1189': 29, '1676': 9, '1677': 3, '876': 10, '747': 3, '656': 2, '1070': 2, '258': 2, '1510': 6, '853': 15, '1502': 2, '617': 2, '320': 3, '886': 2, '843': 3, '515': 7, '209': 2, '971': 2, '236': 2, '1333': 18, '1533': 2, '992': 4, '23': 4, '1336': 2, '198': 2, '1084': 3, '1405': 7, '43': 3, '1613': 3, '1678': 4, '136': 2, '396': 2, '1287': 2, '40': 4, '133': 2, '875': 2, '755': 9, '654': 31, '1288': 14, '608': 4, '542': 2, '1353': 2, '1325': 2, '1082': 32, '679': 3, '551': 2, '1669': 2, '937': 2}
```

Creating Triplets

We use the train and test list to create triplets of (**anchor**, **positive**, **negative**) face data, where positive is the same person and negative is a different person than anchor.

```
[11]: def create_triplets(directory, folder_list, max_files=10):
    triplets = []
    folders = list(folder_list.keys())

    for folder in folders:
        path = os.path.join(directory, folder)
        files = list(os.listdir(path))[:max_files]
        num_files = len(files)

        for i in range(num_files-1):
            for j in range(i+1, num_files):
                anchor = (folder, f"{i}.jpg")
                positive = (folder, f"{j}.jpg")

                neg_folder = folder
                while neg_folder == folder:
                    neg_folder = random.choice(folders)
                neg_file = random.randint(0, folder_list[neg_folder]-1)
                negative = (neg_folder, f"{neg_file}.jpg")
            ((('1580', '3.jpg'), ('1580', '4.jpg'), ('1072', '0.jpg'))
```

```
triplets.append((anchor, positive, negative))
```

```
random.shuffle(triplets)
return triplets
```

```
[12]: train_triplet = create_triplets(ROOT, train_list)
test_triplet = create_triplets(ROOT, test_list)

print("Number of training triplets:", len(train_triplet))
print("Number of testing triplets :", len(test_triplet))

print("\nExamples of triplets:")
for i in range(5):
    print(train_triplet[i])
```

```
Number of training triplets: 9996
Number of testing triplets : 1235
```

```
Examples of triplets:
(('1308', '6.jpg'), ('1308', '7.jpg'), ('736', '0.jpg'))
(('422', '1.jpg'), ('422', '2.jpg'), ('1616', '12.jpg'))
(('1410', '3.jpg'), ('1410', '5.jpg'), ('1400', '1.jpg'))
(('1597', '3.jpg'), ('1597', '4.jpg'), ('495', '1.jpg'))
(('1580', '3.jpg'), ('1580', '4.jpg'), ('1072', '0.jpg'))
```

Creating Batch-Generator

Creating a **Batch-Generator** that converts the triplets passed into batches of face-data and **preprocesses** it before returning the data into separate lists.

Parameters:

- Batch_size: Batch_size of the data to return
- Preprocess: Whether to preprocess the data or not

```
[13]: def get_batch(triplet_list, batch_size=256, preprocess=True):
    batch_steps = len(triplet_list)//batch_size

    for i in range(batch_steps+1):
        anchor = []
        positive = []
        negative = []

        j = i*batch_size
        while j<(i+1)*batch_size and j<len(triplet_list):
            a, p, n = triplet_list[j]
            anchor.append(read_image(a))
            positive.append(read_image(p))
            negative.append(read_image(n))
```

```

anchor = np.array(anchor)
positive = np.array(positive)
negative = np.array(negative)

if preprocess:
    anchor = preprocess_input(anchor)
    positive = preprocess_input(positive)
    negative = preprocess_input(negative)

yield ([anchor, positive, negative])

```

Plotting the Data

Plotting the data generated from `get_batch()` to see the results

```

[14]: num_plots = 6

f, axes = plt.subplots(num_plots, 3, figsize=(15, 20))

for x in get_batch(train_triplet, batch_size=num_plots, preprocess=False):
    a,p,n = x
    for i in range(num_plots):
        axes[i, 0].imshow(a[i])
        axes[i, 1].imshow(p[i])
        axes[i, 2].imshow(n[i])
        i+=1
    break

```



5.2 CNN Model Architecture CODE

```

import numpy as np
import tensorflow as tf
import keras
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

from keras.models import Sequential
from keras.layers import Conv2D, MaxPooling2D, Dense, Flatten, Dropout
import matplotlib.pyplot as plt
from keras.layers.normalization import BatchNormalization
from keras_preprocessing import image
from tensorflow.keras.preprocessing.image import ImageDataGenerator

train_dir = "../input/face-recognition-dataset/Original Images/Original Images/"
generator = ImageDataGenerator()
train_ds = generator.flow_from_directory(train_dir, target_size=(224, 224), batch_size=32)
classes = list(train_ds.class_indices.keys())

```

Found 2562 images belonging to 31 classes.

```

model = Sequential()
model.add(Conv2D(32, kernel_size = (3, 3), activation='relu', input_shape=(224,224,3)))
model.add(MaxPooling2D(pool_size=(2,2)))
model.add(BatchNormalization())
model.add(Conv2D(64, kernel_size=(3,3), activation='relu'))
model.add(MaxPooling2D(pool_size=(2,2)))
model.add(BatchNormalization())
model.add(Conv2D(64, kernel_size=(3,3), activation='relu'))
model.add(MaxPooling2D(pool_size=(2,2)))
model.add(BatchNormalization())
model.add(Conv2D(32, kernel_size=(3,3), activation='relu'))
model.add(MaxPooling2D(pool_size=(2,2)))
model.add(BatchNormalization())
model.add(Conv2D(32, kernel_size=(3,3), activation='relu'))
model.add(MaxPooling2D(pool_size=(2,2)))
model.add(BatchNormalization())
model.add(Dropout(0.2))
model.add(Flatten())
model.add(Dense(128, activation='relu'))
#model.add(Dropout(0.3))
model.add(Dense(len(classes),activation='softmax'))

```

```

model.compile(
    loss = 'categorical_crossentropy',
    optimizer = 'adam',
    metrics = ["accuracy"])
model.summary()

```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
conv2d_10 (Conv2D)	(None, 222, 222, 32)	896
max_pooling2d_10 (MaxPooling)	(None, 111, 111, 32)	0
batch_normalization_10 (Batch Normalization)	(None, 111, 111, 32)	128
conv2d_11 (Conv2D)	(None, 109, 109, 64)	18496
max_pooling2d_11 (MaxPooling)	(None, 54, 54, 64)	0
batch_normalization_11 (Batch Normalization)	(None, 54, 54, 64)	256
conv2d_12 (Conv2D)	(None, 52, 52, 64)	36928
max_pooling2d_12 (MaxPooling)	(None, 26, 26, 64)	0
batch_normalization_12 (Batch Normalization)	(None, 26, 26, 64)	256

conv2d_14 (Conv2D)	(None, 10, 10, 32)	9248
max_pooling2d_14 (MaxPooling)	(None, 5, 5, 32)	0
batch_normalization_14 (Batch Normalization)	(None, 5, 5, 32)	128
dropout_2 (Dropout)	(None, 5, 5, 32)	0
flatten_2 (Flatten)	(None, 800)	0
dense_4 (Dense)	(None, 128)	102528
dense_5 (Dense)	(None, 31)	3999

Total params: 191,455
 Trainable params: 191,007
 Non-trainable params: 448

5.3 Siamese Network Model Code

```

class DistanceLayer(layers.Layer):
    def __init__(self, p=2, **kwargs):
        super().__init__(**kwargs)
        self.p = p

    def call(self, anchor, positive, negative):
        ap_distance = tf.reduce_sum(tf.abs(anchor - positive) ** self.p, axis=-1)
        an_distance = tf.reduce_sum(tf.abs(anchor - negative) ** self.p, axis=-1)
        return (ap_distance, an_distance)

def get_siamese_network_2(input_shape=(128, 128, 3), p=2):
    encoder = get_encoder(input_shape)

    # Input Layers for the images
    anchor_input = layers.Input(input_shape, name="Anchor_Input")
    positive_input = layers.Input(input_shape, name="Positive_Input")
    negative_input = layers.Input(input_shape, name="Negative_Input")

    ## Generate the encodings (feature vectors) for the images
    encoded_a = encoder(anchor_input)
    encoded_p = encoder(positive_input)
    encoded_n = encoder(negative_input)

    # A layer to compute the distances using Minkowski distance
    distances = DistanceLayer(p)(
        encoder(anchor_input),
        encoder(positive_input),
        encoder(negative_input)
    )

    # Creating the Siamese Network model
    siamese_network_2 = Model(
        inputs=[anchor_input, positive_input, negative_input],
        outputs=distances,
        name="Siamese_Network_2"
    )
    return siamese_network_2

siamese_network_2 = get_siamese_network_2(p=2) # You can specify the value of p here
siamese_network_2.summary()

```

Model: "Siamese_Network_2"

Layer (type)	Output Shape	Param #	Connected to
Anchor_Input (InputLayer)	[(None, 128, 128, 3)]	0	
Positive_Input (InputLayer)	[(None, 128, 128, 3)]	0	
Negative_Input (InputLayer)	[(None, 128, 128, 3)]	0	
Encode_Model (Sequential)	(None, 256)	22043944	Anchor_Input[0][0] Positive_Input[0][0] Negative_Input[0][0]
distance_layer_2 (DistanceLayer ((None,), (None,))	0		Encode_Model[3][0] Encode_Model[4][0] Encode_Model[5][0]
Total params: 22,043,944			
Trainable params: 9,583,800			
Non-trainable params: 12,460,144			

5.4 Design Approach and Structure Front-end (frontend/face-similarity-app/)

The frontend is designed as a single-page React application with Tailwind CSS for a responsive, user-friendly interface. The upload page features a clean form with three file inputs for the Anchor, Image 1, and Image 2, styled for clarity and ease of use. A "Submit" button triggers an API request via Axios to the backend. The result page displays the three uploaded images side by side, with their computed Euclidean distances and a clear statement of which image is more similar to the Anchor, ensuring users can quickly interpret the SNN's output.

Upload Page:

- Header: Title ("Face Similarity Detection").
- Form: Three file inputs (Anchor, Image 1, Image 2).
- Button: "Submit" to send images to the backend.

Result Page:

- Section: Display three images (Anchor, Image 1, Image 2).
- Labels: Distances for Image 1 and Image 2.
- Result: Text indicating the more similar image (e.g., "Image 1 is more similar").

The image shows a web application interface for "Face Similarity Detection AI". At the top, the title "Face Similarity Detection AI" is displayed, followed by the instruction "Upload 3 images - AI will determine which is closer to the Anchor Image". Below this, there are three rounded rectangular boxes for uploading images, labeled "Upload Anchor Image", "Upload Image 1", and "Upload Image 2". A blue "Submit" button is positioned below these boxes. At the bottom, a "Results" section displays the text "Image 1 is more similar to the Anchor Image".

Fig. 5.1 UI design of the front-end application

5.4.2 Design Approach and Structure Back-end (api/main.py)

The backend is designed as a lightweight Flask API to process image similarity requests efficiently. It exposes a single `/predict` endpoint that accepts three images, preprocesses them by resizing to 64x64 and normalizing their pixel values, and passes them through the pre-trained SNN model to compute Euclidean distances. The API returns a JSON response containing the distances and a statement identifying the more similar image, ensuring seamless integration with the frontend. Basic error handling validates inputs, and temporary files are cleaned up to maintain efficiency.

API Endpoint:

- `/predict`: POST method to handle image uploads.

Processing Pipeline:

- Image Upload: Receive three images (Anchor, Image 1, Image 2).
- Preprocessing: Resize to 64x64, normalize to $[0, 1]$.
- SNN Prediction: Compute distances using the loaded SNN model.
- Response: JSON with distances and similarity result.
-

Error Handling:

- Validate: Check for missing images.
- Cleanup: Remove temporary image files after processing.

```
1 from fastapi import FastAPI, File, UploadFile
2 from tensorflow.python.keras.models import load_model
3 from io import BytesIO
4 import tensorflow as tf
5 print(tf.__version__)
6 from PIL import Image
7 import uvicorn
8 import h5py
9 import numpy as np
10
11 def check_h5():
12     with h5py.File('model/model_weights.h5', 'r') as file:
13         # Print the keys at the root level of the H5 file
14         print("Root keys:", list(file.keys()))
15         if 'Siamese_Network_2' in file:
16             print("Group keys:", list(file['Siamese_Network_2'].keys()))
17
18 # .h5 file
19 def load_final_model():
20     try:
21         check_h5()
22         model = load_model('model/model_weights.h5')
23         print("Model Loaded")
24         return model
25     except Exception as e:
26         print("Couldn't load model: ", e)
27
28
29
30 # .pb file
31 # Model was trained on tensorflow 2.4.1 and Python 3.8, hence wasn't loading on Python 3.12
32 # Now model is loaded, but the predict function is not working
33 def load_final_model():
34     try:
35         # saved_model = tf.saved_model.load("model/assets/")
36         saved_model = load_model("model/assets/")
37         print("Model loaded")
38         return saved_model
39     except Exception as e:
40         print("Couldn't load model: ", e)
41
42 def img_preprocess(img):
43     # resize to 128x128
44     img = img.resize((128, 128))
45     img = tf.keras.preprocessing.image.img_to_array(img)
46     img = tf.keras.applications.vgg16.preprocess_input(img)
47     img = tf.expand_dims(img, axis=0) # Add batch dimension
48     # TensorSpec(shape=(None, 128, 128, 3), dtype=tf.float32, name='inputs/0')
49     img = tf.convert_to_tensor(img, dtype=tf.float32)
50     img = img / 255.0
51     return img
52
53 def predictSimilarImage(anchor: Image.Image, img1: Image.Image, img2: Image.Image):
54     anchor = img_preprocess(anchor)
55     img1 = img_preprocess(img1)
56     img2 = img_preprocess(img2)
57
58     # anchor = tf.identity(anchor, name="input_image")
59
60     classifier = load_final_model()
61     print("Classifier ", classifier)
62
63     prediction = None
64     try:
65         prediction = classifier.predict([anchor, img1, img2])
66     except Exception as e:
67         print("Couldn't predict: ", e)
68
69     print("prediction ", prediction)
70     return prediction
71
72 # for api
73 def read_imagefile(file) -> Image.Image:
74     img = Image.open(BytesIO(file)) #Pillow object
75     return img
76
77 def file_check(file):
78     extension = file.filename.split(".")[-1] in ("jpg", "jpeg", "png")
79     if not extension:
80         return False
81     return True
82
83
84 app = FastAPI()
85
86 @app.get('/test')
87 async def testing():
88     t = load_final_model()
89     print("Model ", t)
90     return "Hello World"
91
92 @app.post('/predict')
93 async def predict_api(fileAnchor: UploadFile = File(...), file1: UploadFile = File(...), file2: UploadFile = File(...)):
94
95     # extension check
96     if not file_check(fileAnchor):
97         return {"Error": "Image must be jpg or png format!"}
98     if not file_check(file1):
99         return {"Error": "Image must be jpg or png format!"}
100     if not file_check(file2):
101         return {"Error": "Image must be jpg or png format!"}
102
103     anchor = read_imagefile(await fileAnchor.read())
104     img1 = read_imagefile(await file1.read())
105     img2 = read_imagefile(await file2.read())
106
107     try:
108         prediction = predictSimilarImage(anchor, img1, img2)
109     except Exception as e:
110         return {"Error": str(e)}
111
112     final_prediction = dict()
113
114     # prediction is (array([1.579145]), dtype=float32, array([0.], dtype=float32))
115     # convert this to a dictionary
116
117     if prediction[0] <= prediction[1]:
118         final_prediction.update({"Result": "Image 1 is more similar to Anchor Image"})
119     else:
120         final_prediction.update({"Result": "Image 2 is more similar to Anchor Image"})
121
122     # if prediction:
123     #     final_prediction.update({"Result": "Image 1 is more similar to Anchor Image"})
124     # else:
125     #     final_prediction.update({"Label": "Image 2 is more similar to Anchor Image"})
126
127     return final_prediction
128
129 if __name__ == "__main__":
130     uvicorn.run(app, host="127.0.0.1", port=8080)
131
```

Chapter-6

RESULTS

The Results of a project report on "Face similarity detection" encompass a comprehensive evaluation of the proposed approach, aiming to provide insights into its performance, robustness, computational efficiency, generalization ability, ethical considerations, future directions, and real-world applications. This introduction outlines the key areas that will be addressed in the project report, highlighting the significance of each outcome in advancing the field of face recognition technology.

1. Performance Evaluation:

- Quantitative assessment of recognition accuracy achieved by the proposed approach compared to existing methods using standard benchmark datasets like LFW (Labeled Faces in the Wild) or CASIA-WebFace.
- Analysis of precision, recall, F1-score, and other relevant metrics to provide a comprehensive understanding of the model's performance.

2. Robustness Analysis:

- Detailed investigation into the model's robustness against various environmental factors, including variations in lighting conditions, facial expressions, pose, occlusions, and image quality.
- Demonstration of the model's ability to maintain high recognition accuracy in real-world scenarios through experimentation and case studies.
-

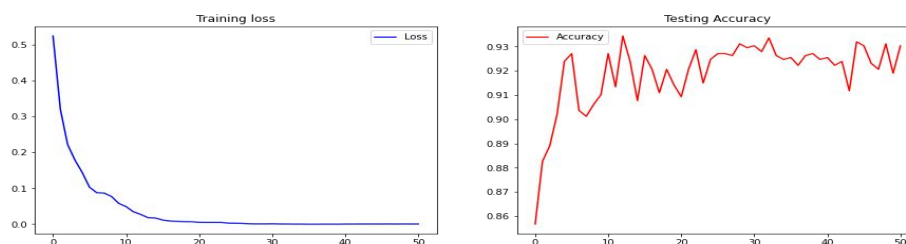
3. Computational Efficiency:

- Evaluation of the computational requirements for both training and inference phases, including memory usage, processing speed, and scalability across different hardware platforms.
- Comparison of the proposed method's efficiency with alternative approaches, highlighting its advantages in terms of resource utilization and runtime performance.

4. Comparison with Baselines:

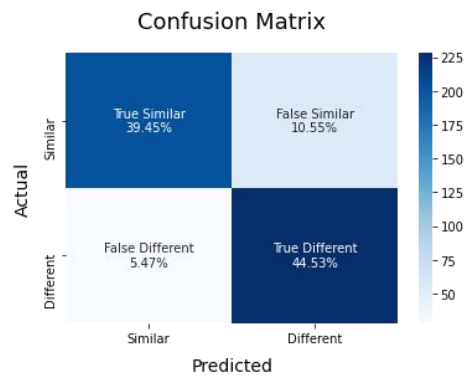
- Comparative analysis of the proposed method with baseline approaches, such as traditional feature-based methods or single-network CNN approaches.
- Highlighting the strengths of the proposed approach in terms of accuracy, robustness, and efficiency, supported by empirical evidence and statistical tests.

5. Siamese Network Results



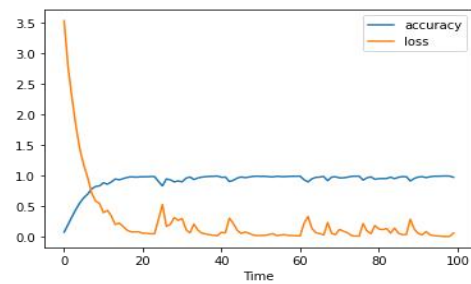
- **Accuracy of Siamese Network Model: 0.93984375**

6. Confusion Matrix

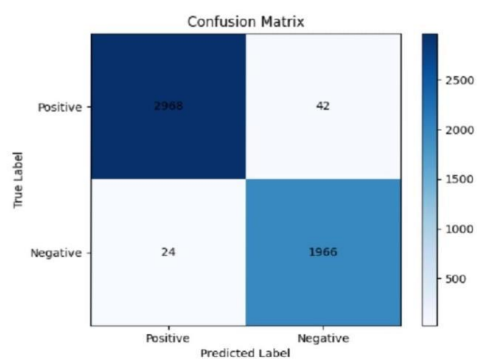


7. CNN Model Outcomes:-

- Accuracy v/s loss analysis.



- Accuracy of CNN Model: 0.97



CHAPTER-7

CONCLUSION

7.1 CONCLUSION

The Face Similarity Detection AI project effectively demonstrated the integration of Convolutional Neural Networks (CNNs) and Siamese Neural Networks (SNNs) to address the problem of determining facial similarity between images. The project was structured into distinct modules to ensure a systematic approach: the **Data Pre-processing Module** prepared the LFW dataset by resizing images to 64x64x3, normalizing pixel values, and splitting the data into training (4274 images), validation (916 images), and test (917 images) sets, saving them as .npy files for efficient access. The **CNN Feature Extraction Module** developed a deep CNN architecture with three convolutional layers (32, 64, and 128 filters), batch normalization, max-pooling, and dense layers to extract 512-dimensional feature vectors, which were trained using categorical cross-entropy loss on 1324 classes and saved as `cnn_feature_extractor.keras`. The **Siamese Network Module** utilized these features in a twin-branch architecture, computing Euclidean distances between image pairs to assess similarity. The **Training Module** optimized the SNN with triplet loss, ensuring the model learned to minimize distances between similar images and maximize distances for dissimilar ones. The **Recognition Module** enabled inference by comparing an anchor image with two others, identifying the closer match. The **Backend Module**, built with FastAPI, provided a `/predict` endpoint to process image uploads, leveraging the CNN and SNN for similarity detection, while the **Frontend Module**, developed using React, Axios, and Tailwind CSS, delivered an intuitive UI for users to upload images and view results. Finally, the **Testing Module** validated the system by verifying data shapes, visualizing CNN training metrics, and ensuring end-to-end functionality via Postman and frontend tests.

Throughout the development, challenges such as memory constraints (due to large image sizes), label encoding errors (unseen labels in validation), and dependency issues (Tailwind CSS setup) were systematically addressed by resizing images, combining labels for encoding, and reinstalling packages. The final system successfully met its objective, accurately determining which of two images is more similar to an anchor image, with the React frontend providing a user-friendly interface and the FastAPI backend ensuring efficient processing. This project not only showcased the power of CNNs for feature extraction and SNNs for similarity comparison but also highlighted the importance of a modular, end-to-end pipeline in building AI-driven applications.

7.2 FUTURE SCOPE

The future scope of the Face Similarity Detection AI includes enhancements to the front-end interface for improved user experience, refinement of the model for more accurate similarity comparisons, exploration of advanced back-end functionalities such as real-time processing, and integration of additional features like facial recognition for broader application possibilities. Additionally, the future scope involves conducting user feedback sessions and usability testing to gather insights for further iteration and enhancement of the Face Similarity Detection AI, ensuring it meets the evolving needs and expectations of its users.

- 1. Advanced CNN Architectures:** Replace the current CNN with state-of-the-art models like ResNet50 or EfficientNet to extract more discriminative features, potentially improving similarity detection accuracy.
- 2. SNN Enhancements:** Experiment with alternative loss functions such as contrastive loss or implement online triplet mining to dynamically select harder triplets during training, enhancing the SNN's ability to differentiate subtle facial features.
- 3. Larger and Diverse Datasets:** Incorporate larger datasets like VGGFace2 or CelebA, which include more diverse facial features and demographic representations, to improve the model's generalization across various ethnicities, ages, and lighting conditions.
- 4. Real-Time Processing:** Optimize the pipeline for real-time applications, such as video-based face similarity detection, by reducing inference time through model quantization or deployment on GPU/TPU hardware.
- 5. Mobile Application Development:** Develop a mobile app using React Native, leveraging the existing React frontend, to make the system accessible on iOS and Android devices for wider user adoption.
- 6. Cloud Deployment and Scalability:** Deploy the FastAPI backend on a cloud platform like AWS or Google Cloud, using containerization (Docker) and orchestration (Kubernetes) to handle high user traffic and ensure scalability.
- 7. Bias Mitigation and Fairness:** Address potential biases in the dataset by ensuring balanced representation across demographics, and implement fairness-aware algorithms to reduce bias in similarity predictions.
- 8. User Interface Enhancements:** Add features to the frontend, such as drag-and-drop image uploads, progress indicators during API calls, and visualization of similarity scores, to improve user experience.
- 9. Multi-Modal Integration:** Extend the system to incorporate additional modalities, such as voice or text-based user inputs, for a more comprehensive identity verification system.
- 10. Security Improvements:** Enhance the backend with authentication mechanisms (e.g., JWT) and secure file handling to protect user-uploaded images and ensure privacy compliance (e.g., GDPR).

REFERENCES

1. V. Patil, A. Narayan, V. Ausekar, A. Dinesh, Automatic students attendance marking system using image processing and machine learning, in: 2020 International Conference on Smart Electronics and Communication (ICOSEC), 2020.
2. S. Kumaar, R.M. Vishwanath, S.N. Omkar, A. Majeedi, A. Dogra, Disguised facial recognition using neural networks, in: 2018 IEEE 3rd International Conference on Signal and Image Processing (ICSIP), 2018.
3. K. Sasirekha, K. Thangavel, Optimization of K-nearest neighbor using particle swarm optimization for face recognition, *Neural Comput. Appl.* 31 (2019).
4. I. Melekhov, J. Kannala, E. Rahtu, Siamese network features for image matching, in: 2016 23rd International Conference on Pattern Recognition (ICPR), 2016, pp. 378–383. Link.
5. Jie Wang, Zihao Li, Research on face recognition based on CNN, *IOP Conf. Ser. Earth Environ. Sci.* 170 (3) (2018).
6. Mustafa H. Mustafa Zuhaer Nayef Al-Dabagh, Mohammed Alhabib, Firas H. ALMukhtar, Face recognition system based on Kernel discriminant analysis, K nearest neighbor and support vector machine, *Int. J. Relig. Educ.* 5 (3) (2018).
7. J.P. Jose, P. Poornima, K.M. Kumar, A novel method for color face recognition using KNN classifier, in: 2012 International Conference on Computing, Communication and Applications, 2012.
8. S. Shinde, M. Shende, J. Shah, H. Shelar, An approach for e-voting using face and fingerprint verification, in: 2020 IEEE Pune Section International Conference (PuneCon), 2020.
9. Xingping Dong, Jianbing Shen, Triplet loss in siamese network for object tracking, *Proc. Eur. Confer. Computer Vis.* (2018) 459–474.
10. E. Jose, G. M, M.T.P. Haridas, M.H. Supriya, Face recognition based surveillance system using FaceNet and MTCNN on Jetson TX2, in: 2019 5th International Conference on Advanced Computing & Communication Systems (ICACCS), 2019.
11. V.K. Verma, V. Kansal, P. Bhatnagar, Patient identification using facial recognition, in: 2020 International Conference on Futuristic Technologies in Control Systems & Renewable Energy (ICFCR), 2020.
12. Shuai Peng, Hongbo Huang, Weijun Chen, Liang Zhang, Weiwei Fang, More trainable inception-ResNet for face recognition, *Neurocomputing* 411 (2020).
13. W. Xing, Y. Bei, Medical health big data classification based on KNN classification algorithm, *IEEE Access* 8 (2019).
14. S. Zhang, X. Li, M. Zong, X. Zhu, R. Wang, Efficient kNN classification with different numbers of nearest neighbors, *IEEE Transact. Neural Networks Learn. Syst.* 29 (5) (2017).
15. Z. Chen, L.J. Zhou, X. Da Li, J.N. Zhang, W.J. Huo, The Lao text classification method based on KNN, *Proc. Comput. Sci.* 166 (2020).
16. S. Kanithan, N.A. Vignesh, E. Karthikeyan, N. Kumareshan, An intelligent energy efficient cooperative MIMO-AF multi-hop and relay based communications for Unmanned Aerial Vehicular networks, *Comput. Commun.* 154 (2020).
17. Minhajur Rahman, Saimunur Rahman, Mohamed Uvaze Ahamed Ayoobkhan, On the effectiveness of deep transfer learning for Bangladeshi meat based curry image classification, in: 2022 International Conference on Innovations in Science, Engineering and Technology (ICISSET), IEEE, 2022.
18. Minhajur Rahman, Saimunur Rahman, Mohamed Uvaze Ahamed Ayoobkhan, Finetuned convolutional neural networks for Bangladeshi vehicle classification, in: 2022 International Conference on Innovations in Science, Engineering and Technology (ICISSET), IEEE, 2022.

18. S.T. Suganthi, Mohamed Uvaze Ahamed Ayoobkhan, Nebojsa Bacanin, K. Venkatachalam, Hubalovský ' St ˇ ep' an, and Trojovský Pavel. "Deep learning model for deep fake face recognition and detection.", PeerJ Computer Science 8 (2022).
19. T. Guo, J. Dong, H. Li, Y. Gao, Simple convolutional neural network on image classification, in: 2017 IEEE 2nd International Conference on Big Data Analysis (ICBDA), IEEE, 2017, March.
20. F. Sultana, A. Sufian, P. Dutta, Advancements in image classification using convolutional neural network, in: 2018 Fourth International Conference on Research in Computational Intelligence and Communication Networks (ICRCICN), IEEE, 2018, November.
21. Y. Pei, Y. Huang, Q. Zou, X. Zhang, S. Wang, Effects of image degradation and degradation removal to CNN-based image classification, IEEE Trans. Pattern Anal. Mach. Intell. 43 (4) (2019).
22. Building FastAPIs with Deep Learning models, an article on Medium by Abdul Wajid Ganiyu. (2024).
23. Deepdive article from deploying deep learning models by Michael Herman on TestDriven.io (2024).
24. Blog post from Nvidia about creating scalable machine learning microservices with python (2023).
25. Deployment of Containerized Machine Learning Model Application on Medium by Srinivas Pratapgiri. (2021)
26. FaceNet: A Unified Embedding for Face Recognition and Clustering:
<https://arxiv.org/abs/1503.03832>
27. Image similarity estimation using a Siamese Network with a triplet loss:
https://keras.io/examples/vision/siamese_network/
28. Celebrity
Face Recognition:
<https://www.kaggle.com/ravehgillmore/celebrityfacerecognition/>
29. Face Recognition using Siamese Networks:
<https://medium.com/wicds/face-recognition-using-siamese-networks-84d6f2e54e>